

Localo UI Foundation Blueprint

UI documentation, package foundation, theme setup, API client foundation, and shared UI component plan

Prepared for Businice Developers - Localo

Purpose

This document captures the agreed UI direction for Localo. It intentionally focuses only on the first UI foundation stages so the UI can be built with the same discipline used for the backend: planned, phased, reviewable, and easy for Codex to execute without wasting tokens.

Current backend checkpoint

- Backend phases 0-9 are complete, tested, linted, committed, and pushed.
- UI should now start from foundation and standards, not random screens.
- No Admin, Shop Owner, or Public Web screen should be built until UI-0 to UI-4 are finished and reviewed.

1. Agreed UI Foundation Scope

For now, the agreed UI work covers only these foundation stages:

Phase	Name	Goal	Not included
UI-0	UI Documentation	Create UI docs, standards, structure, theme rules, and implementation plan.	No code changes except docs.
UI-1	Foundation Packages	Prepare packages, aliases, index exports, and folder foundation.	No real screens and no business components.
UI-2	Theme Setup	Create Localo base theme with Admin, Shop Owner, and Public Web override support.	No app-specific UI screens.
UI-3	API Client Foundation	Create typed API client, error handling, token helpers, response unwrap, and pagination foundations.	No direct feature screens.
UI-4	Shared UI Components	Build generic UI primitives and layout building blocks for all apps.	No Localo business components like ShopsTable or ProductForm.

Important principle

This stage prepares the UI system. It does not build Admin screens, Shop Owner screens, or Public Web screens.

2. Final UI Work Order

The broader UI roadmap remains phase-based, but this document locks only the foundation stage. The agreed order is:

1. UI-0: UI Documentation
2. UI-1: Foundation Packages
3. UI-2: Theme Setup
4. UI-3: API Client Foundation
5. UI-4: Shared UI Components

- 6. UI-5: Admin App Shell
- 7. UI-6: Admin Auth/Login
- 8. UI-7: Admin Shops
- 9. UI-8: Admin Categories
- 10. UI-9: Shop Owner App Shell
- 11. UI-10: Shop Owner Profile
- 12. UI-11: Shop Owner Products
- 13. UI-12: Support + Commission UI
- 14. UI-13: Notifications + Audit Logs UI
- 15. UI-14: Public Web Discovery

Why this order works

- Admin creates and approves shops first.
- Shop Owner manages the approved shop profile and products next.
- Public Web discovery comes later, after real shop and product data exist.
- Theme, API client, and shared UI components must exist before app screens to avoid duplicated one-off UI.

3. UI Codebase Structure

The UI should use app-specific feature folders and shared packages. Apps own screens and feature behavior. Packages own reusable primitives, theme, utilities, and API access.

```
localo/
├── apps/
│   ├── admin/
│   ├── shop-owner/
│   └── web/
├── packages/
│   ├── ui/
│   ├── theme/
│   ├── api-client/
│   ├── shared-types/
│   ├── config/
│   └── utils/
├── docs/
│   ├── ui/
│   │   ├── UI_PROJECT_STRUCTURE.md
│   │   ├── UI_CODE_STANDARDS.md
│   │   ├── UI_THEME_GUIDE.md
│   │   └── UI_IMPLEMENTATION_PLAN.md
│   ├── PROJECT_STRUCTURE.md
│   ├── AI_CODING_RULES.md
│   └── LOCALO_BACKEND_TASKS.md
```

4. UI-0: UI Documentation

UI-0 creates the documentation that Codex must follow before any UI code is generated.

Document	Purpose
docs/ui/UI_PROJECT_STRUCTURE.md	Defines app folders, package folders, feature folder pattern, templates, aliases, and index.ts rules.
docs/ui/UI_CODE_STANDARDS.md	Defines component standards, form standards, table standards, naming, API rules, auth rules, and Codex rules.
docs/ui/UI_THEME_GUIDE.md	Defines Localo base theme, app theme variants, semantic colors, status colors, and token rules.
docs/ui/UI_IMPLEMENTATION_PLAN.md	Defines UI phase order, phase boundaries, acceptance checks, and Codex prompts.

UI-0 acceptance checklist

- docs/ui folder exists.
- All four UI documents are created.
- Docs clearly say UI-0 to UI-4 are foundation only.
- Docs mention templates, aliases, index.ts exports, package boundaries, and no business screens yet.
- No frontend screens are built in this phase.

5. UI-1: Foundation Packages

UI-1 prepares package structure, path aliases, index exports, and basic package metadata. It should not build actual business screens or feature components.

Package	Responsibility
packages/theme	Localo design tokens, app themes, Tailwind preset, CSS variable strategy.
packages/ui	Reusable generic UI primitives, form fields, layout building blocks, feedback, navigation helpers.
packages/api-client	Typed API client, base HTTP wrapper, token helpers, response unwrap, API errors.
packages/shared-types	Shared enums and API types. Must not import React.
packages/config	Shared config and environment helpers when needed.
packages/utils	Generic utilities like cn, date, currency, string, and pagination helpers.

Required alias pattern

```
@localo/ui
@localo/theme
@localo/api-client
@localo/shared-types
@localo/config
@localo/utils
```

Index export rule

- Every package must have src/index.ts.
- Feature folders may have index.ts for app-local clean imports.
- Export only stable public APIs from index.ts.
- Avoid deep relative imports across the repo.

Good:

```
import { Button } from '@localo/ui';
import { localoThemes } from '@localo/theme';
import { shopsApi } from '@localo/api-client';
```

Bad:

```
import { Button } from '../../../packages/ui/src/primitives/button/button';
```

6. UI-2: Theme Setup

UI-2 creates one shared Localo theme system. By default, all apps use the same theme, but the structure must allow Admin, Shop Owner, and Public Web to diverge easily later.

```
packages/theme/
├── src/
│   ├── index.ts
│   └── tokens.ts
```

```

├─ colors.ts
├─ typography.ts
├─ spacing.ts
├─ radius.ts
├─ shadows.ts
├─ app-themes.ts
└─ tailwind-preset.ts

```

Theme	Initial behavior	Future flexibility
localoBaseTheme	Source of truth for all apps.	Can be extended with brand changes.
adminTheme	Initially same as base.	Can become more dashboard/data-table focused.
shopOwnerTheme	Initially same as base.	Can become warmer and action-focused.
publicWebTheme	Initially same as base.	Can become more discovery/customer focused.

Theme rules

- Do not hardcode random colors inside components.
- Use theme tokens, semantic colors, or UI component variants.
- Include semantic colors: primary, background, surface, text, muted, border, success, warning, danger, info.
- Include status colors for shops, products, support tickets, payments, subscriptions, notifications, and audits.
- Theme package should not depend on React unless a provider is absolutely needed later.

7. UI-3: API Client Foundation

UI-3 creates the frontend API access layer. No page or component should directly call fetch. This package should hide backend response shape, token handling, errors, and pagination.

```

packages/api-client/
├─ src/
│   ├─ index.ts
│   ├─ http/
│   │   ├─ http-client.ts
│   │   ├─ api-error.ts
│   │   ├─ auth-token.ts
│   │   └─ response.ts
│   ├─ auth/
│   │   ├─ auth.api.ts
│   │   └─ auth.types.ts
│   ├─ shops/
│   ├─ categories/
│   ├─ products/
│   ├─ support-tickets/
│   ├─ commission/
│   ├─ notifications/
│   └─ audit-logs/

```

API client responsibilities

- Base API URL configuration.
- Access token handling and refresh-token helper foundation.
- Backend response unwrap for success/data/meta/error.
- Normalized API error class.
- Pagination query and meta types.
- Typed module APIs such as authApi, shopsApi, productsApi.
- No direct fetch in app pages, templates, components, or hooks.

Good:

```
const shops = await shopsApi.list(filters);
```

```
Bad:
const res = await fetch('/admin/shops');
```

8. UI-4: Shared UI Components

UI-4 builds generic reusable components only. These components live in packages/ui and must not know Localo business concepts like Shop, Product, Commission, or Support Ticket.

```
packages/ui/
├── src/
│   ├── index.ts
│   ├── primitives/
│   ├── form-fields/
│   ├── layout/
│   ├── feedback/
│   ├── navigation/
│   └── utils/
```

Form fields

```
packages/ui/src/form-fields/
├── input/
├── textarea/
├── select/
├── multi-select/
├── checkbox/
├── switch/
├── radio/
├── search-input/
├── password-input/
├── number-input/
├── email-input/
├── phone-input/
├── date-input/
├── time-input/
├── file-input/
└── form-field/
```

Layout components

```
packages/ui/src/layout/
├── sidebar/
├── topbar/
├── dashboard-shell/
├── page-header/
├── page-container/
└── section/
```

Generic sidebar components

- Sidebar
- SidebarHeader
- SidebarContent
- SidebarFooter
- SidebarGroup
- SidebarItem
- SidebarLink
- SidebarCollapseButton

Sidebar rule

packages/ui provides generic sidebar building blocks. App-specific menu items must live inside apps/admin or apps/shop-owner later.

Shared component build order

16. utils/cn
17. Button
18. Badge
19. Card
20. FormField wrapper
21. Input, Textarea, Select, MultiSelect
22. Checkbox, Switch, Radio
23. SearchInput, PasswordInput, NumberInput, EmailInput, PhoneInput
24. DateInput, TimeInput, FileInput
25. Dialog and ConfirmDialog
26. Table primitives
27. EmptyState, ErrorState, LoadingState
28. Spinner and Skeleton
29. Toast
30. Pagination
31. PageHeader and PageContainer
32. Sidebar, Topbar, DashboardShell

Not allowed in packages/ui

- ShopStatusBadge
- ProductStatusBadge
- ShopsTable
- ProductForm
- CommissionSummaryCard
- SupportTicketDetail
- AdminSidebarMenu
- ShopOwnerSidebarMenu
- PublicShopCard

9. Page, Template, and Feature Standards

We agreed to call page-level composition components templates. Next.js route files should stay thin and render templates.

apps/admin/src/app/shops/page.tsx

```
import { ShopsTemplate } from '@features/shops';

export default function ShopsPage() {
  return <ShopsTemplate />;
}
```

```
apps/admin/src/features/shops/
├── templates/
│   ├── shops-template.tsx
│   ├── shop-detail-template.tsx
│   └── create-shop-template.tsx
├── components/
│   ├── shops-table.tsx
│   ├── shop-form.tsx
│   ├── shop-filters.tsx
│   └── shop-status-badge.tsx
├── hooks/
├── schemas/
└── utils/
```

Template rule

- Route page = Next.js route file.
- Template = page-level screen composition.
- Feature components = tables, forms, filters, cards, dialogs.
- UI primitives = generic reusable building blocks from packages/ui.
- Pages should not contain large business logic or API calls.

10. UI Code Standards

Area	Standard
Components	Use generic primitives in packages/ui and feature-specific components in app feature folders.
Forms	Use React Hook Form + Zod for important forms. Include validation, submit loading, API errors, and success state.
Tables	Use consistent search, filters, pagination, loading, empty, error, and row action patterns.
Server state	Use TanStack Query for API-backed data.
API calls	All API calls go through packages/api-client. No direct fetch in components.
Theme	Use packages/theme tokens and UI variants. Avoid random hardcoded colors.
Auth	Route protection and role-based UI. Backend remains source of truth.
Naming	Files use kebab-case. Components use PascalCase. Hooks start with use.
Imports	Use aliases and index.ts exports. Avoid deep relative imports.
Codex	One phase or screen at a time. No full UI build in one task.

11. Codex Execution Rules for UI Foundation

- Read docs/ui docs before coding UI.
- Implement only the requested UI phase.
- Do not touch backend unless explicitly asked.
- Do not create Admin, Shop Owner, or Public Web screens during UI-0 to UI-4.
- Do not create business components in packages/ui.
- Do not install random UI libraries without checking package.json and explaining why.
- Do not duplicate package exports or create deep import paths.
- Run relevant typecheck/lint/test commands after each phase if available.
- At the end, summarize files changed, commands run, checks, assumptions, and pending items.

12. Phase Acceptance Checklist

Phase	Acceptance checks
UI-0	docs/ui files created; standards captured; no UI screens or app code added.
UI-1	packages exist; aliases configured; index.ts exports present; no business components.
UI-2	theme package has base theme, app themes, tokens, Tailwind preset/CSS variable strategy.
UI-3	api-client has http client, response unwrap, auth token helpers, error handling, pagination types.
UI-4	packages/ui has generic primitives, form fields, layout, feedback, navigation, sidebar building blocks.

13. Suggested Codex Prompt Pattern

Use this pattern for each UI foundation phase. Replace the phase name and requirements as needed.

```
Read docs/AI_CODING_RULES.md.
Read docs/PROJECT_STRUCTURE.md.
Read docs/ui/UI_PROJECT_STRUCTURE.md.
Read docs/ui/UI_CODE_STANDARDS.md.
Read docs/ui/UI_THEME_GUIDE.md.
Read docs/ui/UI_IMPLEMENTATION_PLAN.md.
```

Implement UI-X only: <phase name>.

Rules:

- Do not build Admin, Shop Owner, or Public Web screens.
- Do not touch backend code.
- Do not create Localo business components in packages/ui.
- Use aliases and index.ts exports.
- Keep changes scoped to this phase.
- Run relevant checks.

At the end, summarize completed work, files changed, commands run, checks, assumptions, and pending items.

14. Final Decision Summary

- We will handle UI like backend: planned, phased, and reviewed before commit.
- UI-0 to UI-4 are foundation phases only.
- Theme setup happens before app screens to avoid hardcoded UI.
- API client foundation happens before feature screens to avoid direct fetch duplication.
- Shared UI components happen before Admin Shell so the shell uses reusable primitives.
- Page-level components are called templates.
- Every package and feature should use aliases and index.ts exports.
- packages/ui/form-fields will hold Input, Textarea, Select, MultiSelect, Checkbox, Switch, Radio, SearchInput, PasswordInput, NumberInput, EmailInput, and related fields.
- packages/ui/layout/sidebar will hold generic sidebar building blocks.
- Business-specific UI components will live inside app feature folders, not packages/ui.